

Semantic Runtime Protocol: Governing Semantic Runtime State through Validated Boundaries and Evidence-Controlled Optimization

SRP Authors

Abstract

Semantic systems increasingly maintain evolving semantic runtime states that combine representational content, provenance, and transition authority, yet existing approaches often optimize retrieval, storage, or action generation without explicitly governing when a semantic transition is admissible. We present Semantic Runtime Protocol (SRP), a governance framework that treats semantic state evolution as a controlled transition problem. SRP separates observation, validation, optimization, evidence, governance, and execution. It first identifies evaluated feasible regions for semantic transitions under explicit invariants and then performs constrained optimization inside those regions to generate governed recommendations rather than direct mutations. SRP also supports evidence-controlled verification by allowing stronger semantic evidence to refine decisions without increasing execution authority. Across the evaluated semantic workloads and runtime contracts, SRP evaluates whether semantic evolution can be represented with measurable transition variables, bounded by evaluated constraints, reconstructed through governed implementations, and audited through explicit evidence records. These results suggest that, under evaluated runtime contracts, semantic runtime evolution can be represented as a governed transition process in which mutation authority remains explicit and separable from evidence quality and optimization objectives.

1 Semantic Runtime Protocol: Governing Semantic Runtime State through Validated Boundaries and Evidence-Controlled Optimization

1.1 Abstract

Semantic systems increasingly maintain evolving semantic runtime states that combine representational content, provenance, and transition authority, yet existing approaches often optimize retrieval, storage, or action generation without explicitly governing when a semantic transition is admissible. We present Semantic Runtime Protocol (SRP), a governance framework that treats semantic state evolution as a controlled transition problem. SRP separates observation, validation, optimization, evidence, governance, and execution. It first identifies evaluated feasible regions for semantic transitions under explicit invariants and then performs constrained optimization inside those regions to generate governed recommendations rather than direct mutations. SRP also supports evidence-controlled verification by allowing stronger semantic evidence to refine decisions without increasing execution authority. Across the evaluated semantic workloads and runtime contracts, SRP evaluates whether semantic evolution can be represented with measurable transition variables, bounded by evaluated constraints, reconstructed through governed implementations, and audited through explicit evidence records. These results suggest that, under evaluated runtime contracts, semantic runtime evolution can be represented as a governed transition process in which mutation authority remains explicit and separable from evidence quality and optimization objectives.

1.2 1. Introduction

Semantic systems increasingly maintain evolving runtime states rather than static representations. In practice, however, systems that operate over semantic state often conflate three different concerns. Evidence is

used as if it were authority, optimization is treated as if it were execution, and adaptation is allowed before the boundary of safe change has been established. The result is a system that may improve local performance while leaving open the question of whether semantic change is still governed.

This paper asks a simple question: how can semantic state evolve only within validated, governed boundaries? SRP answers by governing the admissibility of semantic state transitions, rather than only improving retrieval or generation after a state has already been committed. Observation discovers which variables matter. Validation determines which regions are safe. Optimization ranks candidates only inside those regions. Evidence strengthens verification when uncertainty remains. Governance remains the only layer that can authorize execution.

Related systems provide useful building blocks, while SRP frames a distinct governance problem. Retrieval and memory systems improve access to information, but they do not define transition authority. Agentic systems can plan and act, but they often integrate evidence, decision, and execution within one loop. Reinforcement learning can adapt policies, but SRP asks a prior question about where adaptation is allowed under validated boundaries. SRP is therefore positioned differently: it is a governed semantic transition framework rather than a retrieval system, a memory system, or an adaptive agent.

SRP is also model-agnostic: it can govern semantic transitions produced by LLMs, embedding-based systems, symbolic systems, or hybrid architectures, but it does not depend on any one proposal mechanism to define its core claim.

The paper makes four contributions. First, it introduces semantic runtime state as a governed object of study: semantic content, provenance, and authority are tracked together rather than collapsed into a single opaque memory vector. Second, it provides evidence that evaluated feasible regions can be frozen and then used to constrain optimization, so recommendation remains separate from execution. Third, it provides evidence that additional semantic evidence can strengthen verification without transferring authority. Fourth, it introduces an auditable evaluation protocol for semantic transition governance under frozen runtime contracts.

This paper should be read as a governance framework for semantic state transition systems rather than as a new memory architecture or retrieval algorithm. The recovery experiments presented here are implementation instances used to evaluate SRP under a frozen runtime contract, not the definition of SRP itself.

1.3 2. Related Work

SRP is related to retrieval-based systems, memory systems, autonomous agents, reinforcement learning, constrained optimization, and controlled systems, but it is not reducible to any one of them.

Retrieval and memory systems focus on how to store, compress, and recover information. Their primary question is typically about access quality: what should be retrieved, and how faithfully can it be reconstructed? SRP uses evidence and recovery, but its primary question is different. SRP asks whether a semantic transition is permitted at all, and under what validated conditions that transition may proceed.

Agentic systems emphasize planning, tool use, and autonomous execution. Their strength is action generation, but that strength also creates a risk: observation, decision, and execution may become entangled. SRP separates those layers. Evidence may inform a decision, optimization may recommend a configuration, but governance is the only layer that approves execution.

Reinforcement learning and other adaptive systems optimize policies over time. That setting is valuable when the objective and the action space are already well defined. SRP focuses on a prior question: before a system learns to adapt, can it first validate where adaptation is allowed? In other words, SRP treats governance as a prerequisite to adaptation rather than a consequence of it.

Constrained optimization and formal methods are also relevant because SRP uses boundaries, invariants, and approval gates. The difference is that SRP applies those ideas to evolving semantic runtime state rather than to a static action space.

1.3.1 2.1 Positioning of SRP

SRP is best read as a framework for semantic state transition governance rather than as a new memory module, retrieval pipeline, agent loop, or policy learner.

Approach	Stores semantic state	Retrieves information	Generates actions	Governs semantic transition
Retrieval / RAG	partial	✓	optional	×
Memory systems	✓	✓	optional	×
Agent systems	partial	✓	✓	partial
Reinforcement learning	policy state		✓	action-level only
SRP	✓	✓	optional	✓

This positioning is the novelty boundary of the paper: SRP does not replace these systems; it makes explicit the governance layer that determines when semantic state may change.

1.3.2 2.2 Concrete Prior Systems

Recent memory and agent systems show why semantic state handling matters in practice. Memory-oriented systems such as MemGPT, Letta, Mem0, and Graphiti focus on persistence, retrieval, and reconstruction. Agentic systems such as ReAct, Toolformer, and Voyager focus on reasoning, tool use, and action generation. Safety-oriented optimization methods such as safe RL and constrained RL focus on limiting the action space while preserving policy improvement.

SRP is different from each of these lines of work. It does not ask only how to retrieve, summarize, or act. It asks when a semantic transition is admissible, and which authority layer may approve it. The paper therefore treats existing systems as useful instantiations or baselines for recovery and validation, rather than as direct equivalents of SRP.

1.3.3 2.3 Runtime Verification and Controlled State Transition

SRP shares the separation between validation and execution found in runtime verification and transaction systems, but it applies that separation to semantic runtime state rather than to a fixed program state or a database row. In conventional control paths, validation precedes commit or execution. In SRP, validation precedes authorization, and authorization precedes semantic transition.

This distinction matters because semantic runtime state combines content, provenance, and authority metadata. SRP therefore treats transition admissibility as the central object of control, rather than assuming that a validated representation is already authorized to mutate. The system borrows the discipline of invariant checking and controlled commit boundaries, but the object under control is semantic state evolution.

Database transaction systems provide commit and rollback guarantees over structured state. SRP differs because semantic mutations are judged not only by consistency, but by evidence-bounded admissibility and mutation authority. Classical belief revision studies rational change of knowledge states, while SRP focuses on runtime operational authority governing when semantic mutations may enter an active system state.

The formal governance model is defined in Method, where semantic runtime state, admissibility, and evidence-authority separation are specified explicitly.

1.3.4 2.4 Scope of Claims

To avoid overclaiming, the paper uses evaluated settings language throughout. The experiments support claims about the tested workloads, frozen runtime contracts, and evidence packages prepared under the review boundary. They do not establish universal optimality or a universal semantic governance law.

1.4 3. Method

1.4.1 3.1 Semantic Runtime State and Governed Transition

A semantic runtime state is not defined solely by its representational content. In SRP, a runtime semantic state is modeled as:

$$S_t = (C_t, P_t, A_t)$$

where C_t denotes semantic content maintained at runtime, P_t denotes provenance and supporting evidence associated with the content, and A_t denotes the runtime authority context governing admissible

modifications, whether represented as internal state or externally referenced governance metadata depending on implementation.

A semantic transition is represented as:

$$\text{tau}_t = (\text{S}_t, \text{Delta}_t, \text{E}_t, \text{Gamma}_t, \text{S}_{(t+1)})$$

where Delta_t is a proposed semantic change, E_t is the available evidence associated with the proposal, and Gamma_t represents the governance context, including validation and authorization conditions.

Unlike conventional update mechanisms that directly apply proposed changes, SRP separates verification from authorization:

$$\begin{aligned} V(\text{S}_t, \text{Delta}_t, \text{E}_t) &\rightarrow v \\ G(\text{S}_t, \text{Delta}_t, v, \text{Gamma}_t) &\rightarrow \{0,1\} \end{aligned}$$

where V produces a verification result from the proposal and its evidence, $G = 1$ indicates that the transition satisfies the required governance conditions, and $G = 0$ indicates that the transition is rejected.

The state evolution rule is therefore:

$$\begin{aligned} \text{S}_{(t+1)} &= \\ &\begin{cases} T(\text{S}_t, \text{Delta}_t) & \text{if } G(\text{S}_t, \text{Delta}_t, V(\text{S}_t, \text{Delta}_t, \text{E}_t), \text{Gamma}_t) = 1 \\ \text{S}_t & \text{otherwise} \end{cases} \end{aligned}$$

This formulation separates three concepts that are often conflated in semantic systems:

1. Proposal generation: producing a candidate transition Delta_t .
2. Evidence evaluation: assessing whether the transition is sufficiently supported.
3. Authority governance: determining whether the transition is allowed to modify runtime state.

Consequently, additional evidence may improve transition verification without increasing transition authority. SRP treats evidence availability and mutation authority as independent variables.

This is the core governance property of SRP: semantic evidence may improve verification, but it does not bypass authority.

1.4.2 3.2 Runtime State Representation

Let theta denote the parameter configuration and let e denote evidence. SRP models the transition as:

$$\text{S}_{(t+1)} = T(\text{S}_t, \text{theta}, \text{e})$$

The validated feasible region is:

$$F = \{ \text{theta} \mid \text{invariant}(\text{theta}) = \text{true} \}$$

The optimization stage searches within that region:

$$\text{theta}^* = \text{argmax}_{\{\text{theta} \in F\}} U(\text{theta})$$

but theta^* is a governed recommendation rather than a direct runtime mutation.

We distinguish three functions:

$$R(\text{S}_t, \text{theta}, \text{e}) \rightarrow r$$

produces a recommendation,

$$G(\text{S}_t, r) \rightarrow a$$

produces an authorization decision, and

$$T(\text{S}_t, a) \rightarrow \text{S}_{(t+1)}$$

performs execution when authorized. In SRP, recommendation and authorization are not equivalent. The operational semantics are:

```
T(S_t, theta, e) =
  S_(t+1)    if invariant(theta) = true and G(S_t, R(S_t, theta, e)) = approve
  S_t        otherwise
```

This makes the governance property explicit: failed admissibility checks preserve the current runtime state.

1.4.3 3.3 Controlled Transition Pipeline

SRP follows a governed pipeline:

```
Observation
  |
  v
Validation
  |
  v
Optimization
  |
  v
Evidence
  |
  v
Governance
  |
  v
Execution
```

The order matters: observation discovers what can be measured, validation freezes what can be changed safely, optimization ranks candidates inside that frozen region, evidence refines verification when uncertainty remains, and governance is the approval boundary.

1.4.4 3.4 Authority Separation

SRP assigns each layer a different responsibility:

Component	Authority
Calibration	observe
Validation	verify
Optimization	recommend
Evidence	inform
Governance	approve
Runtime	execute

The central design rule is that recommendation is not execution. More evidence is not more authority. Validation does not mutate the system; it defines the region in which mutation may later be considered.

This gives SRP a clear separation between feasibility and preference: validation determines where optimization may operate, the optimization stage determines which candidate is preferred under a declared objective, and governance determines whether the preferred candidate becomes an actual transition.

Formally, a transition is admissible only if:

```
theta in F
and
G(S_t, R(S_t, theta, e)) = approve
```

Otherwise, the runtime state is preserved.

1.4.5 3.5 Algorithmic Core

SRP’s main algorithm can be written as a two-stage control loop. The first stage discovers and freezes the feasible region; the second stage recommends and governs transitions inside that region.

Algorithm SRP-Transition

Input: semantic state S_t , candidate parameters θ , evidence sources E

Output: governed transition decision or advisory recommendation

1. Observe current semantic runtime state S_t
2. Validate candidate region F using invariants and closure checks
3. If $\theta \notin F$:
 reject mutation; return advisory failure
4. Rank candidates inside F with objective $U(\theta)$
5. Evaluate additional evidence according to predefined verification policy
6. Produce governed recommendation $r = \operatorname{argmax}_{\{\theta \in F\}} U(\theta)$
7. If governance approves r :
 execute transition and update runtime state
 else:
 preserve state and return rejected recommendation

The boundary discovery step and the governance step are intentionally separate: boundary discovery answers whether a change may be considered, and governance answers whether the change may be executed.

SRP’s external validation follows the same pattern:

Algorithm SRP-External-Validation

Input: benchmark slice B , baseline set M , frozen runtime contract K

Output: evidence package and promotion decision

1. Freeze benchmark slice, runtime contract, and shared evaluation protocol
2. Run all methods in M under K
3. Compute official metrics and SRP diagnostics
4. Audit scorer parity, attribution, and failure boundaries
5. Record evidence manifest and descriptive statistics
6. Promote the evidence package only if audit gates pass

This keeps evaluation reproducible and prevents benchmark scoring from silently changing the method being evaluated. Recovery is treated as one possible transition implementation under SRP, rather than the definition of SRP itself.

1.4.6 Protocol Invariant 1: Authority Independence

For any semantic transition τ_t , increasing evidence availability E_t may change verification and approval outcomes, but it does not imply an increase in authority level A_t .

Rationale: In SRP, authority level is determined by Γ_t , while evidence contributes to the verification result V . Since E_t and A_t are modeled as independent components of the transition tuple, changes in evidence quality cannot directly elevate authority without an explicit governance rule.

1.4.7 3.6 Design Properties

SRP is designed to satisfy three practical properties under the frozen evaluation contract:

- Boundary safety: runtime mutation is only considered after a candidate passes the validated feasible region.
- Authority separation: evidence and recommendation can change ranking, but they do not increase execution authority.

- **Replayability:** the same runtime contract, scorer, and freeze settings should reproduce the same audit trail.

These are design properties, not universal proofs, but they are the reasons SRP can be audited as a governed protocol rather than treated as an unconstrained adaptation loop.

The key transition-preservation property is:

$$G(S_t, R(S_t, \theta, e)) = \text{reject} \Rightarrow S_{t+1} = S_t$$

In other words, rejected transitions preserve runtime state.

1.4.8 3.7 Governance Properties

SRP can also be stated as a small set of protocol-level properties.

Property 1: Transition safety.

$$G(S_t, R(S_t, \theta, e)) = \text{reject} \Rightarrow T(S_t, \theta, e) = S_t$$

Rejected transitions cannot modify semantic runtime state.

Property 2: Authority non-escalation.

$$\text{Auth}(G(S_t, R(S_t, \theta, e_1))) = \text{Auth}(G(S_t, R(S_t, \theta, e_2)))$$

unless governance policy explicitly changes. Stronger evidence can improve verification confidence, but it does not increase execution authority.

Property 3: Recommendation-execution separation.

$$R(S_t, \theta, e) = r \\ \text{not} \Rightarrow T(S_t, r) = S_{t+1}$$

A recommendation is not a transition. A valid transition still requires governance approval.

Property 4: Replay consistency.

$$\text{SRP}(S, K) = \text{SRP}(S, K)$$

under an identical state representation, evidence package, governance policy, and frozen runtime contract. These properties make SRP a protocol rather than a loose architecture description.

1.4.9 3.8 Figures and Positioning Summary

The paper should include two figures that carry the main narrative visually.

Figure 1: Semantic Runtime Protocol transition governance pipeline. This figure should show the end-to-end transition order:

Observation -> Validation -> Optimization -> Evidence -> Governance -> Execution

The visual should make the authority split explicit: observation measures, validation freezes, optimization recommends, evidence informs, governance approves, and runtime executes only after approval. The key message is that recommendation and execution are separate.

Proposal Sources

LLM | Embedding | Symbolic

|

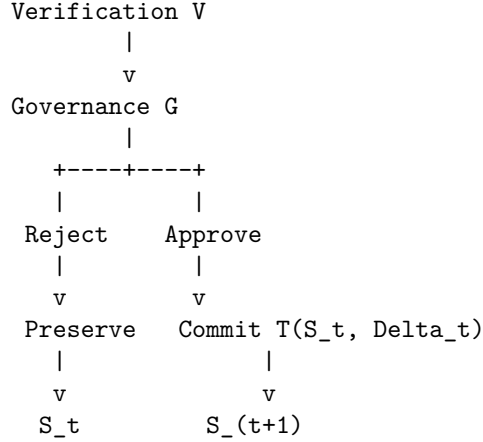
v

Candidate Semantic Transition

(Delta_t, E_t)

|

v



The figure should visually separate verification from authorization and commit: stronger evidence can improve V , but only G can admit the transition into runtime state and mutate S_t .

Figure 2: Positioning of SRP by semantic state and transition authority abstractions. This figure should contrast SRP with retrieval, memory, agent, and reinforcement-learning systems according to their primary roles and authority abstractions. The goal is not to claim those systems are wrong, but to illustrate that SRP asks a different question:

How should semantic runtime state transition be governed?

The figure should make the novelty boundary visible by mapping SRP to semantic runtime state transitions and governance authority, while illustrating retrieval, memory, agent, and RL as systems with different primary foci and authority models.

1.5 4. Experiments

The experiments are designed to evaluate SRP as a semantic governance framework rather than to establish universal superiority over memory or retrieval systems. External systems are used as reference implementations where applicable, while controlled validations and artifact-backed evaluations provide the primary evidence for SRP properties.

The paper applies the same separation principle to its own evaluation process: evidence generation, evidence validation, and claim promotion are treated as distinct stages.

The experiments are organized into four layers. First, core governance validation checks whether SRP can observe state, identify evaluated boundaries, constrain optimization, enforce authority separation, verify protocol properties, and reject injected invalid transitions. Second, implementation validation studies evaluate how SRP behaves when instantiated through reconstruction and parameter settings. Third, robustness studies test whether the governance semantics remain consistent across the evaluated workloads, representations, and storage backends. Fourth, external validation checks whether the same frozen contract can support paper-facing evidence support.

The detailed density sweep, boundary generalization study, and full LongMemEval calibration traces are documented in the appendix and review report rather than repeated here.

The experimental boundary is fixed:

- runtime implementation remains fixed
- no online learning
- no autonomous mutation
- optimization outputs are advisory
- evidence backends do not control execution

The research questions are grouped as follows:

Layer	Question
Governance	Can SRP observe semantic transition variables, identify validated boundaries, constrain optimization
Implementation	Can governed reconstruction preserve structure, and are recommendations stable under repeated
Robustness	Do governance semantics remain stable across workloads, representations, and storage backends?
External validation	Can the frozen contract support calibration, evidence promotion, and scorer alignment under an

1.5.1 4.1 Core Governance Validation

The main governance chain is:

Evaluation	Main Result
Semantic observability	130 transition observations, replay success 1.0, state consistency 1.0
Boundary validation	10 / 25 feasible candidates, stable extents across densities
Constrained optimization	60% search reduction with the same top candidate as the naive full-grid sweep
Evidence-controlled governance	verification improves from 0.50 to 1.00 while authority remains unchanged; invalid t

Semantic observability collects repeated transition observations over the frozen parameter axes `activation_threshold`, `recovery_min_evidence`, `preserve_evidence`, and `archive_relations`. The measurements indicate that the transition variables are explicit and reproducible before optimization decisions are introduced.

Boundary validation uses invariant checking, closure validation, and replay equivalence to identify the evaluated feasible region. The main result is a stable boundary over the evaluated density conditions. The detailed density sweep and boundary generalization checks are placed in the appendix.

Constrained optimization compares SRP with a naive full-grid sweep over the same candidate space. SRP reaches the same top objective while reducing the search budget by 60 percent, which indicates that the validated boundary is operationally useful.

The evidence-controlled governance check merges the previous verification and boundary-enforcement slices. Vector evidence plus semantic evidence improves verification quality, and the authority-violation slice still ends with a zero final accept rate. In short, evidence can improve verification, but it does not override governance in the evaluated settings.

Protocol property verification closes the core governance chain by checking transition safety, authority non-escalation, recommendation-execution separation, and replay consistency under the frozen contract. Negative transition injection provides the complementary negative test: invalid transitions remain rejected under full SRP, while weakened variants accept some injected cases and therefore expose the need for the governance gate.

The core governance chain can be summarized as follows:

Validation Objective	Metric / Check	Result
Semantic observability	Transition observations, replay success, state consistency	130, 1
Boundary validation	Feasible candidates, stable extents across densities	10 /
Constrained optimization	Search reduction with the same top candidate	60% re
Evidence-controlled governance	Verification confidence, authority change	0.50
Protocol property verification	Property checks under frozen contract	4 / 4
Negative transition injection	Invalid accept rate under full SRP / no gate / evidence-as-authority	0.00

This table is the paper’s compact governance summary: it shows observability, boundary control, optimization constraint, evidence-authority separation, protocol-level verification, and negative testing in one view.

1.5.2 4.2 Implementation Case Study: Governed Reconstruction

4.2.1 Purpose Recovery is not part of the definition of SRP. It is a case study for transition validation: a semantic reconstruction process must decide which information survives a state transition, and SRP constrains that decision through explicit governance boundaries.

In that sense, semantic reconstruction is itself a transition:

`S_t -> compression / reconstruction -> S_(t+1)`

SRP does not redefine reconstruction as the core contribution. It uses reconstruction to evaluate whether governed transitions preserve structure under evidence, provenance, and authority constraints.

4.2.2 Evaluation Setup This evaluation asks whether a semantic reconstruction process improves structural preservation when transition decisions are constrained by explicit governance boundaries.

It compares three recovery modes:

- vector-only recovery
- vector plus relation expansion
- relation-closure recovery

The recovery observables in this case study align with SRP properties as follows:

SRP Property	Recovery Observable
Evidence preservation	Closure Acc.
Authority boundary	Hallucinated Relation Rate
State consistency	Drift
Semantic retention	Relation Acc.

4.2.3 Results The main result is that the SRP-governed recovery implementation improves structural fidelity under the same recovery budget:

Mode	Mean Relation Acc.	Mean Closure Acc.	Mean Drift	Mean Hallucinated Rel. Rate
vector_only	0.333333	0.166667	0.433333	0.0
relation_expansion	0.875	0.8125	0.145834	0.3125
relation_closure	0.875	0.8125	0.083333	0.0

The summary view is:

Recovery Mode	Coverage	Relation Acc.	Closure Acc.	Drift	Hallucinated Relation
Vector-only	0.392857	0.333333	0.166667	0.433333	0.0
Relation expansion	0.738095	0.875	0.8125	0.145834	0.3125
Relation closure	0.738095	0.875	0.8125	0.083333	0.0

4.2.4 Interpretation The phase indicates that relation-aware recovery changes the preservation profile, but it remains an implementation instance rather than the framework definition. The key point is not that recovery is the primary contribution, but that SRP-controlled transition constraints shape how semantic structure is preserved during reconstruction.

1.5.3 4.3 Transition Configuration Sensitivity

This evaluation asks how SRP parameters move the system across fidelity-cost tradeoff regions while keeping the recovery strategy fixed.

The main pattern is that different parameters move the evaluated operating point across different parts of the observed tradeoff surface:

- `archive_relations` improves relation fidelity and reduces drift, but increases cost
- `preserve_evidence` improves provenance stability and slightly narrows drift, but adds cost
- `relation_depth` has the largest structural effect, with deeper recovery improving coverage and closure at higher cost
- `activation_threshold` shifts the acceptance boundary, trading coverage against drift in a smoother way

The measured sweep makes the pattern concrete:

Setting
Baseline (<code>archive_relations=False</code> , <code>preserve_evidence=False</code> , <code>relation_depth=1</code> , <code>activation_threshold=0.9</code>)
<code>archive_relations=True</code>
<code>preserve_evidence=True</code>
<code>relation_depth=0</code>
<code>relation_depth=2</code>
<code>relation_depth=3</code>
<code>activation_threshold=0.1</code>

The recommendation-stability result is folded into this section: the recommendation was fully consistent across 10 seeds under the frozen workload, objective, and evidence backend. That stability result is the baseline that makes the sensitivity sweep interpretable.

1.5.4 4.4 Robustness

This evaluation asks whether SRP preserves its governance semantics across the evaluated semantic workloads rather than only on the SRP-shaped prototype.

The robustness study combines cross-workload, representation, and backend checks under the same governance pipeline.

Cross-workload validation compares code memory, knowledge reasoning, and agent planning under the same relation-aware recovery baseline:

Domain	Coverage	Drift	Relation Acc.	Closure Acc.	Hallucinated Rel. Rate
Code memory	0.638889	0.227778	0.666667	0.666667	0.138889
Knowledge reasoning	0.500000	0.400000	0.500000	0.250000	0.333333
Agent planning	0.465278	0.355556	0.500000	0.416667	0.111111

The normalized comparison against vector-only recovery is:

Domain	Vector-only Closure	Relation-closure Closure	Delta
Code memory	0.000000	1.000000	+1.000000
Knowledge reasoning	0.000000	0.375000	+0.375000
Agent planning	0.000000	0.625000	+0.625000

Representation robustness indicates stable hierarchy and governance consistency across the tested encoders and parsers:

Metric	Value
Cases evaluated	144
Hierarchy consistency rate	1.0
Governance consistency rate	1.0

Backend robustness indicates the same stability across storage backends:

Metric	Value
Cases evaluated	36
Hierarchy consistency rate	1.0
Governance consistency rate	1.0

Taken together, these evaluations provide evidence that SRP’s governance semantics remained consistent across the tested workloads, representations, and storage backends.

1.5.5 4.5 External Evidence Package Validation

The external validation work is split into calibration and evidence.

LoCoMo was used as a calibration boundary to validate the adapter, semantic translation layer, and attribution protocol. It is treated as calibration artifact, not as final paper evidence.

LongMemEval was prepared as an external validation evidence package under a frozen shared local-vLLM runtime contract. The evidence package includes:

- full context
- sliding window
- vector RAG
- Mem0
- Graphiti
- Letta
- MemMachine
- SRP

The evidence package keeps the runtime manifest frozen, reports descriptive statistics for the fixed slice, co-reports official scores and SRP diagnostics, and documents scorer alignment and promotion decisions explicitly. It therefore serves as paper-facing external validation support under a frozen evaluation scope rather than as a claim of universal benchmark dominance.

1.6 5. Discussion

SRP is not a memory system, a RAG extension, or an autonomous agent. Its core claim is more specific: semantic runtime governance is a first-class systems problem.

1.6.1 5.1 Applicability to LLM-based Semantic Systems

Semantic Runtime Protocol is designed as a model-agnostic governance layer for semantic state transitions. SRP does not assume a specific proposal mechanism or representation model. A semantic transition may be generated by an LLM agent, an embedding-based retrieval system, a symbolic reasoning component, or a hybrid architecture. The role of SRP is not to produce candidate transitions, but to determine whether a proposed transition is admissible before it becomes part of the runtime semantic state.

In LLM-based systems, an LLM can act as a transition proposer by generating candidate updates from conversations, observations, tool interactions, or external information sources. SRP introduces an intermediate governance boundary between proposal generation and state mutation:

LLM Proposal -> SRP Validation -> Semantic State Update

This separation distinguishes semantic generation from semantic authority. A generated proposal, regardless of model confidence or supporting evidence, does not automatically obtain permission to modify runtime state. Instead, SRP evaluates transition validity through explicit validation conditions, evidence constraints, and authority rules.

Therefore, SRP should not be interpreted as an LLM memory architecture or retrieval optimization method. Its objective is to govern semantic state evolution across different underlying representation and generation mechanisms. The current evaluation uses controlled semantic representations to isolate governance properties, while LLM-generated transitions represent a compatible future deployment scenario rather than a prerequisite for the framework itself.

This framing explains the experimental pattern: semantic state variables can be observed, feasible regions can be identified and frozen, constrained optimization can operate inside an evaluated region without becoming a control mechanism, and evidence escalation can improve verification without transferring authority. The broader evaluations provide evidence that the same governance semantics remained consistent across the evaluated workloads, representations, and implementations. Recovery and reconstruction remain important implementation cases, but the framework claim is about governing semantic transitions.

The external validation results reinforce the same lesson. Calibration and evidence are separated. Scorer alignment is treated as an auditable boundary. The evidence package is accepted only under a frozen contract, not because the system is universally optimal.

1.7 6. Limitations and Future Work

The current SRP baseline does not implement autonomous semantic adaptation, online learning, self-modifying runtime behavior, or automatic policy updates. The constrained optimization stage produces a recommendation, not a learned policy.

The validated feasible region is workload dependent and objective dependent. SRP therefore claims a governed feasible region for the evaluated setting, not a universal boundary for all semantic workloads.

The current paper does not fully measure local model latency, energy cost, or deployment overhead. The tradeoff between verification gain and evidence cost remains an important future concern.

Boundary discovery currently uses parameter sampling, invariant checking, and closure validation. Larger or higher-dimensional spaces may require adaptive sampling, surrogate modeling, or formal verification support.

Finally, governance authority is assumed to exist outside the runtime. SRP does not resolve who defines governance or how governance policies are negotiated.

Future work should focus on provenance-aware governance, confidence calibration, candidate pruning, and broader benchmark coverage. Those extensions belong to a future version of SRP, not to the frozen evidence chain reported here.

1.8 7. Conclusion

SRP provides a governance-first framework for semantic transition. It separates observation, validation, optimization, evidence, governance, and execution so that semantic state can change only within evaluated boundaries. The experiments provide evidence that semantic transition variables can be observed, feasible regions can be identified and frozen, optimization can be constrained, evidence can improve verification without transferring authority, and SRP-controlled transitions can preserve semantic structure through the evaluated recovery implementations and workloads.

The main paper-facing claim is therefore not that SRP universally solves memory or adaptation. It is that semantic runtime governance is a first-class control problem, and semantic evolution can be represented as measurable, bounded, auditable, and governable under a frozen evaluation contract in the evaluated settings. Recovery and reconstruction are examples of SRP-controlled behaviors, not the framework’s primary definition.

1.9 References (selected)

This list is intentionally selective. It covers the prior-work anchors that define SRP’s novelty boundary and the core control models it builds on.

- Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" (2020): <https://arxiv.org/abs/2005.11464>
- Packer et al., "MemGPT: Towards LLMs as Operating Systems" (2023): <https://arxiv.org/abs/2310.08560>
- Chhikara et al., "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory" (2025): <https://arxiv.org/abs/2504.19413>
- Rasmussen et al., "Zep: A Temporal Knowledge Graph Architecture for Agent Memory" (2025); Graphiti is the temporal knowledge graph engine used by Zep: <https://arxiv.org/abs/2501.13956>
- Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models" (2022): <https://arxiv.org/abs/2210.03629>
- Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools" (2023): <https://arxiv.org/abs/2302.04724>
- Wang et al., "Voyager: An Open-Ended Embodied Agent with Large Language Models" (2023): <https://arxiv.org/abs/2305.16291>
- Achiam et al., "Constrained Policy Optimization" (2017): <https://arxiv.org/abs/1705.10528>
- Wachi et al., "A Survey of Constraint Formulations in Safe Reinforcement Learning" (2024): <https://arxiv.org/abs/2402.04724>
- Haerder and Reuter, "Principles of Transaction-Oriented Database Recovery" (1983): <https://dl.acm.org/doi/10.1145/281861.281862>
- Alchourron, Gardenfors, and Makinson, "On the Logic of Theory Change: Partial Meet Contraction and Revision Functions" (1985): <https://www.jstor.org/stable/2274239>
- Sanchez et al., "A Survey of Challenges for Runtime Verification from Advanced Application Domains (Beyond Software)" (2019): <https://link.springer.com/article/10.1007/s10703-019-00337-w>

1.10 Appendix A. Evidence Provenance and Artifact Mapping

This appendix defines the paper’s reproducibility boundary. It explains which artifacts are trusted, which are supporting evidence, and which remain provenance only in the current release snapshot.

1.10.1 A.1 Evidence Promotion Policy

The default trust policy is conservative:

- artifacts without a `metadata.json` are untrusted
- artifacts with `generated_at` before 2026-07-16T00:00:00-04:00 are legacy-only evidence
- artifacts with `generated_at` on or after the cutoff may be used for claims once the review method passes

The review report is generated from `experiments/results` and records both trusted and legacy artifacts.

1.10.2 A.2 Artifact Classification

The paper uses three levels of artifact status:

- **Main:** directly supports a paper-facing claim
- **Appendix:** useful supporting evidence or legacy evidence that should not carry the main claim alone
- **Archive:** preserve for provenance only

The current snapshot contains one trusted main-claim artifact:

- `interaction_boundary_enforcement`

The current snapshot also contains supporting artifacts that remain appendix-grade:

- `phase_ii_boundary` (compatibility alias for `admissibility_boundary_validation`)
- `external_validation_longmemeval_evidence_strong_baselines`

1.10.3 A.3 Claim-to-Evidence Mapping

The detailed claim-to-evidence mapping is maintained in `audit/CLAIM_EVIDENCE_MAP.md`. That document should be treated as the claim ledger for the current release branch.

In brief:

- authority separation is the strongest currently trusted claim
- feasible-region validation is appendix-supported
- LongMemEval is external-validation support, not refreshed main evidence
- cross-workload robustness and component-level ablation remain under-supported in the current snapshot

1.10.4 A.4 Reproduction Entry Points

The recommended regeneration flow is:

- `python experiments/sensitivity/run_phase_i_observability.py`
- `python -m experiments.sensitivity.phase_i_observability`
- `python -m experiments.validation.admissibility_boundary_validation.runner`
- `python -m experiments.validation.phase_ii_density_baseline`
- `python -m experiments.validation.phase_ii_boundary_generalization`
- `python -m experiments.optimization.phase_iii_governed_optimization.baseline`
- `python -m experiments.optimization.phase_iii_governed_optimization.objective_sensitivity`
- `python experiments/sensitivity/run_activation_recovery_interaction.py`
- `python -m experiments.evaluation.semantic_backend_comparison.runner`
- `python -m experiments.evaluation.phase_v_retention.runner`
- `python -m experiments.evaluation.transition_reconstruction_validation.runner`
- `python -m experiments.evaluation.configuration_stability_validation.runner`
- `python -m experiments.evaluation.configuration_sensitivity_validation.runner`
- `python -m experiments.evaluation.cross_domain_validation.runner`
- `python -m experiments.evaluation.representation_invariance_validation.runner`
- `python -m experiments.evaluation.implementation_independence_validation.runner`
- `python -m experiments.external_validation.manual_sanity`
- `python -m experiments.external_validation.calibration_report`
- `python -m experiments.external_validation.longmemeval_adapter_validation`
- `python -m experiments.external_validation.evidence`
- `python -m experiments.external_validation.scorer_alignment_audit`

1.10.5 A.5 Release Snapshot Limitations

At the time of this draft, the review report marks `interaction_boundary_enforcement` as trusted, while the older phase and external-validation artifacts remain legacy-only because they either lack `metadata.json` or predate the trust cutoff.

The practical reading is:

- `interaction_boundary_enforcement` is admissible for paper-facing claims
- earlier phase artifacts remain provenance material, not claim material
- LongMemEval remains appendix-supported until its snapshot is refreshed and its missing files are reconciled

Legacy entry compatibility:

- `phase_vii_parameter_stability -> configuration_stability_validation`
- `phase_vii_parameter_sensitivity -> configuration_sensitivity_validation`
- `phase_viii_representation_invariance -> representation_invariance_validation`
- `phase_viii_implementation_independence -> implementation_independence_validation`
- `phase_viii_cross_domain -> cross_domain_validation`

This appendix does not add a new claim about SRP. It documents the evidence-management procedure used to separate trusted outputs from legacy outputs in this release branch.

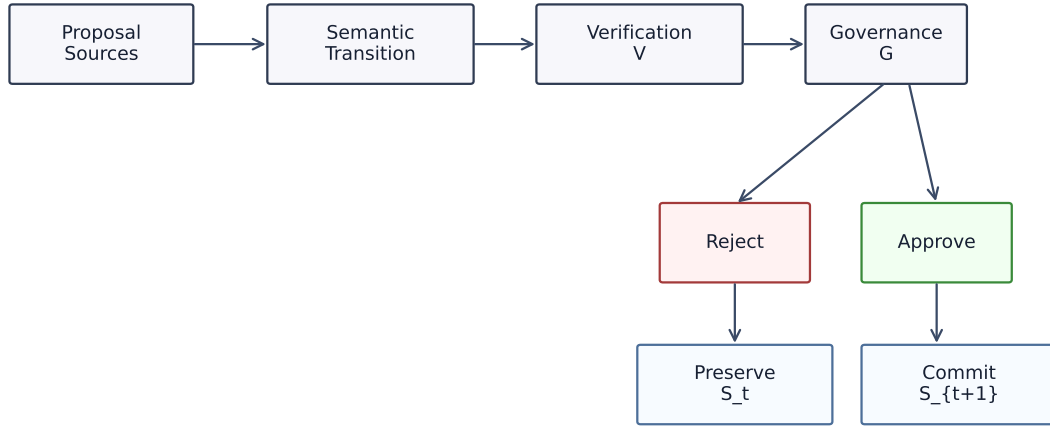
A Artifact Attachment Package

This appendix shell documents the submission-facing packaging boundary. Curated evidence bundles live under `artifacts/`, and the manifest is `artifacts/MANIFEST.md`.

B Figure Assets

The package stores vector figure assets in `figures/`. The assets are packaging aids derived from the manuscript's existing governance narrative and do not add new claims.

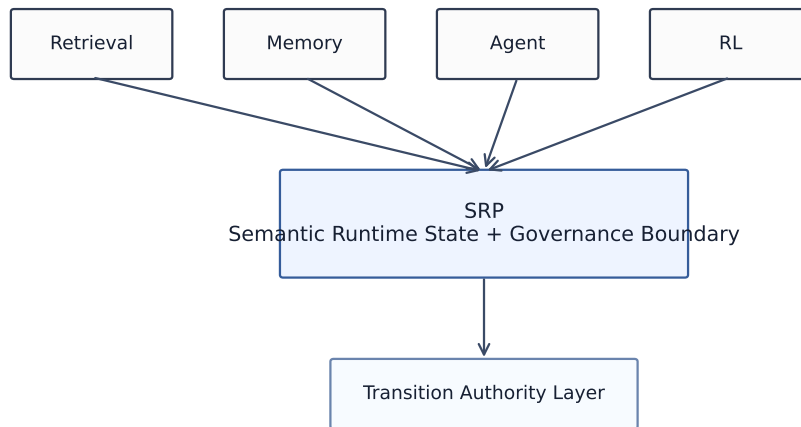
SRP Governance Pipeline



Evidence improves verification; governance controls mutation.

Figure 1: SRP governance pipeline preview for submission packaging.

SRP Positioning by Transition Authority



SRP occupies the transition governance layer, not the retrieval or action layer.

Figure 2: SRP positioning preview for submission packaging.